

Desenvolvimento Web 2

Aula 7

Single Page Applications com React

l.bertholdo@ifsp.edu.br

Conteúdo

- Single Page Application
- Framework React
- JSX – JavaScript XML
- Ferramentas de Desenvolvimento
- DOM e React.DOM
- Criação de Componentes com React
- Ciclo de Vida dos Componentes

Single Page Application



- Não é de hoje que a **experiência de usuário** passou a ser uma preocupação para as empresas que fornecem serviços web.
- No entanto, ainda existem usuários perdendo tempo esperando uma página ser recarregada cada vez que uma nova informação é solicitada.
- Muitas vezes, essa lentidão ocorre porque a página requisitada é recuperada do servidor e seu conteúdo é **inteiramente** carregado no navegador.
- Uma solução para esse problema, bastante considerada atualmente no desenvolvimento web, é utilizar o conceito de **Single Page Application (SPA)**, ou **aplicação de página única**.

***Single Page Application (SPA)** é uma aplicação web que interage com o usuário atualizando dinamicamente partes e dados da página, sem a necessidade de recarregar todo o conteúdo durante seu uso, fazendo com que a aplicação responda de forma mais eficiente ao usuário.*

Single Page Application

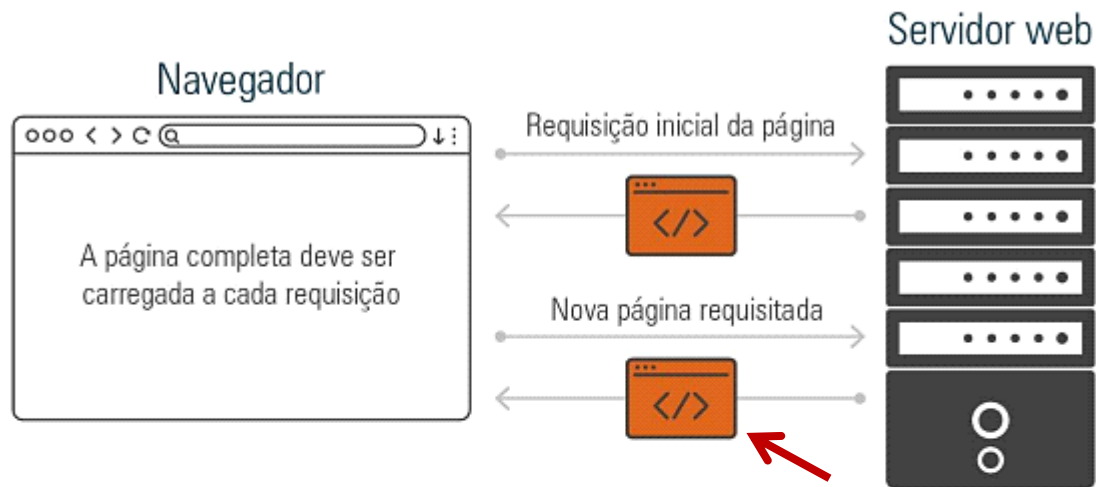


- O conceito de SPA foi lançado no início dos anos 2000 e propôs uma nova maneira de projetar páginas web.
- Em aplicações SPA, o conteúdo exibido no navegador não é uma página HTML completa e estática, e sim partes do **DOM (Document Object Model)** que vão se juntando para construir a interface da aplicação.
- Uma vez que a página inicial é carregada, todos os recursos necessários para criar e exibir um novo conteúdo já estão na interface do usuário, que é a responsável por controlar as transições dos conteúdos na página.

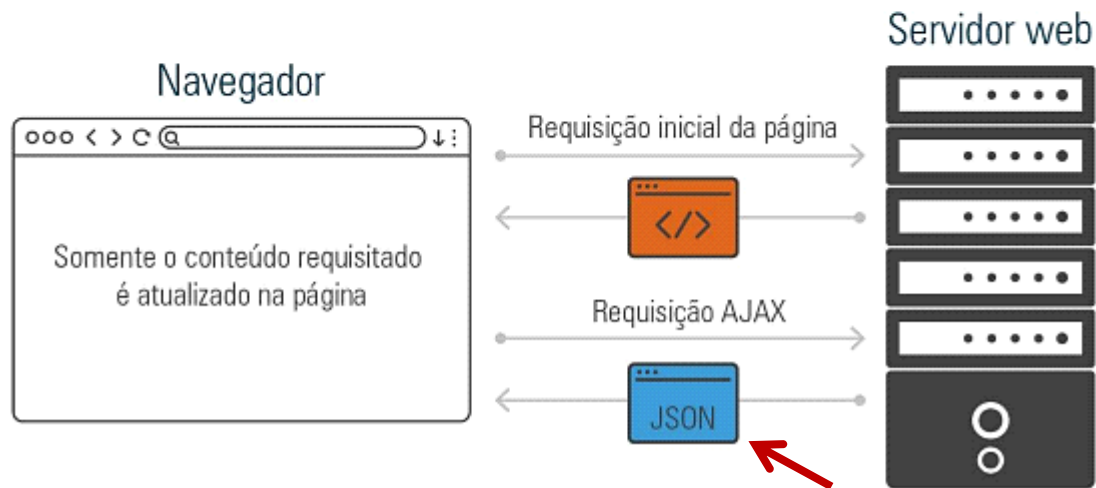
*Em uma SPA, toda nova atualização é processada **localmente** no navegador e anexada dinamicamente ao DOM por meio de JavaScript, e, quando necessário, o servidor envia apenas os **dados brutos** requisitados, geralmente em formato JSON. Isso torna o processo de exibição da página mais rápido, pois o servidor passa a ter menos processos para executar e há menos tráfego na rede ao realizar trocas de dados entre cliente e servidor.*

Single Page Application

a) Páginas tradicionais



b) Páginas SPA



*Diferentemente das aplicações web tradicionais, que precisam renderizar todos os elementos (barra de navegação, textos, rodapés, anúncios etc) sempre que uma página é solicitada, a SPA carrega o novo conteúdo na página sem precisar recarregar todos os seus outros elementos. Isso é possível porque a **camada de apresentação** do website é gerenciada pelo **navegador** e não pelo **servidor**.*

Single Page Application

- Devido ao ganho de performance, com o tempo, as aplicações SPA se tornaram cada vez mais populares. Para auxiliar na criação desse tipo de aplicação, surgiram diversos *frameworks* baseados em JavaScript, entre eles: o Backbone, o Angular, o Vue e o React.

*O **Backbone**, desenvolvido no início de 2010, foi uma das primeiras soluções para os problemas existentes antes do surgimento do React, entre eles a reutilização de código e a testabilidade. Era uma solução elegantemente simples: uma biblioteca que fornecia uma maneira de criar “modelos” e “views”, a partir de uma abordagem própria do padrão de arquitetura MVC (Model View Controller).*

*O **Angular**, lançado pelo Google em 2010, foi outro framework JavaScript pioneiro. Ele teve um impacto significativo no cenário do desenvolvimento web, pois introduziu recursos inovadores, cujas influências podem ser vistas nas bibliotecas que surgiram mais tarde, incluindo o React.*

*O **Vue**, desenvolvido em 2014 durante um dos projetos do Google Creative Labs, permite criar interfaces de usuário e aplicações web SPA. É considerado um framework “progressivo”, pois pode ser utilizado em partes de uma aplicação existente ou para criar, do zero, uma aplicação completa.*

Framework React



- Atualmente, um dos *frameworks* mais populares usados para o desenvolvimento de aplicações SPA é o **React**, também chamado de **React.js** ou **ReactJS**.
- **React** é uma biblioteca JavaScript *open source* criada pelo Facebook em 2013. É utilizada no desenvolvimento *front-end* para simplificar e agilizar a tarefa de construção de interfaces de usuário interativas e de alto desempenho.
- Atualmente, o **React** é mantido pela Meta, e também por uma enorme comunidade de desenvolvedores pelo mundo. Sua documentação oficial encontra-se no site <https://react.dev/>.

*Em 2015, o Facebook lançou também a biblioteca **React Native**, para criação de aplicações móveis que utilizam as funcionalidades nativas das plataformas iOS e Android.*

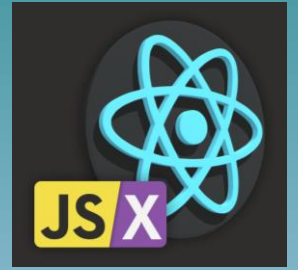
Framework React



- Usar o React para simplesmente criar e renderizar a marcação HTML de uma SPA não é a principal vantagem desta biblioteca. O verdadeiro ganho está no **dinamismo** e na **performance** da aplicação ao usá-la.
- Dada a natureza da JavaScript, qualquer modificação, por mais simples que seja, propaga-se por toda a estrutura da **árvore DOM**, podendo causar desde pequenas instabilidades até travamentos da página.
- Para entregar uma experiência de usuário melhor, criar aplicações de alta performance e rápida renderização, o **React** foi projetado para trabalhar com o **DOM Virtual**, uma abstração da **árvore DOM**.

*Todas as atualizações na interface são aplicadas e processadas no **DOM Virtual** e, apenas depois, são aplicadas em pontos específicos da **árvore DOM**, sem interferir com o restante da árvore do documento.*

JSX – JavaScript XML



- React usa uma sintaxe muito simples e clara denominada **JSX**, uma extensão de sintaxe para JavaScript que simplifica a criação de componentes e facilita a escrita de JavaScript.
- A JSX define cada componente como uma **unidade independente e mínima** que compõe a interface. Isso acelera o processo de desenvolvimento e manutenção, pois os componentes gerados são reusáveis.
- Arquivos contendo sintaxe JSX devem ser gravados com extensão **js**, ou, opcionalmente, **jsx**, pois são arquivos JavaScript.

*React usa o transpiler **Babel** para fazer a “tradução” (transpilação) de JSX para JavaScript, o qual já vem configurado e pronto para uso na biblioteca.*

JSX – JavaScript XML

- Sintaxe JavaScript para criar um elemento **<h1>**:

```
let elemento = document.createElement("h1");  
let titulo = document.createTextNode("Livro Sapiens");  
const resultado = elemento.appendChild(titulo);
```

Resultado: <h1>Livro Sapiens</h1>

- Sintaxe JSX para criar um elemento **<h1>**:

```
const resultado = <h1>Livro Sapiens</h1>;
```

Resultado: <h1>Livro Sapiens</h1>

- Sintaxe JSX para inserção de atributos em elementos HTML:

```
let meuId = "topo";  
const elemento = <div id = { meuId }>Cabeçalho</div>;
```

Resultado: <div id="topo">Cabeçalho</div>

JSX – JavaScript XML

- Em JSX, todas as *tags* devem ser fechadas, ou com a **tag de fechamento tradicional** (`<div></div>`, `<p></p>`), ou com a barra da **tag de fechamento automático** (`<div />`, `<p />`), que inclui *tags* autofechadas (`<br />`, `<img />`).
- Nomes de atributos HTML devem seguir a sintaxe **camelCase**. Ex.: `colSpan`, `onClick`, `onSubmit`.
- Além disso, os atributos HTML **class** e **for** devem ser escritos **className** e **htmlFor**, pois todas as palavras reservadas da JavaScript têm sua sintaxe modificada em JSX.
- Pode-se inserir qualquer **expressão** JavaScript na sintaxe JSX, desde que seja envolvida por chaves. Ex.: `{ expressão JavaScript }`.

*A sintaxe JSX não aceita **declarações** JavaScript entre as chaves, somente **expressões**. Expressões retornam um valor (por exemplo: `x + 3`) e declarações executam uma ação (por exemplo: um bloco `if-else`).*

JSX – JavaScript XML

- Em JSX, a aplicação de **estilos inline** pode ser feita de duas formas:

Definindo e aplicando um objeto contendo estilização *inline* (diretamente):

```
<p style = { { color:"red", fontSize:"28px", textTransform:"uppercase" } }>  
  Texto estilizado inline  
</p>
```

Definindo e aplicando um objeto contendo estilização *inline* (por referência):

```
let estilosInline = {  
  color:"red",  
  fontSize:"28px",  
  textTransform:"uppercase"  
}
```

```
<p style = { estilosInline }>Texto estilizado inline</p>
```

*Em JSX, **propriedades CSS** com mais de uma palavra devem ser escritas com sintaxe **camelCase**. Ex.: backgroundColor. Já os **valores das propriedades** devem ser definidos entre aspas.*

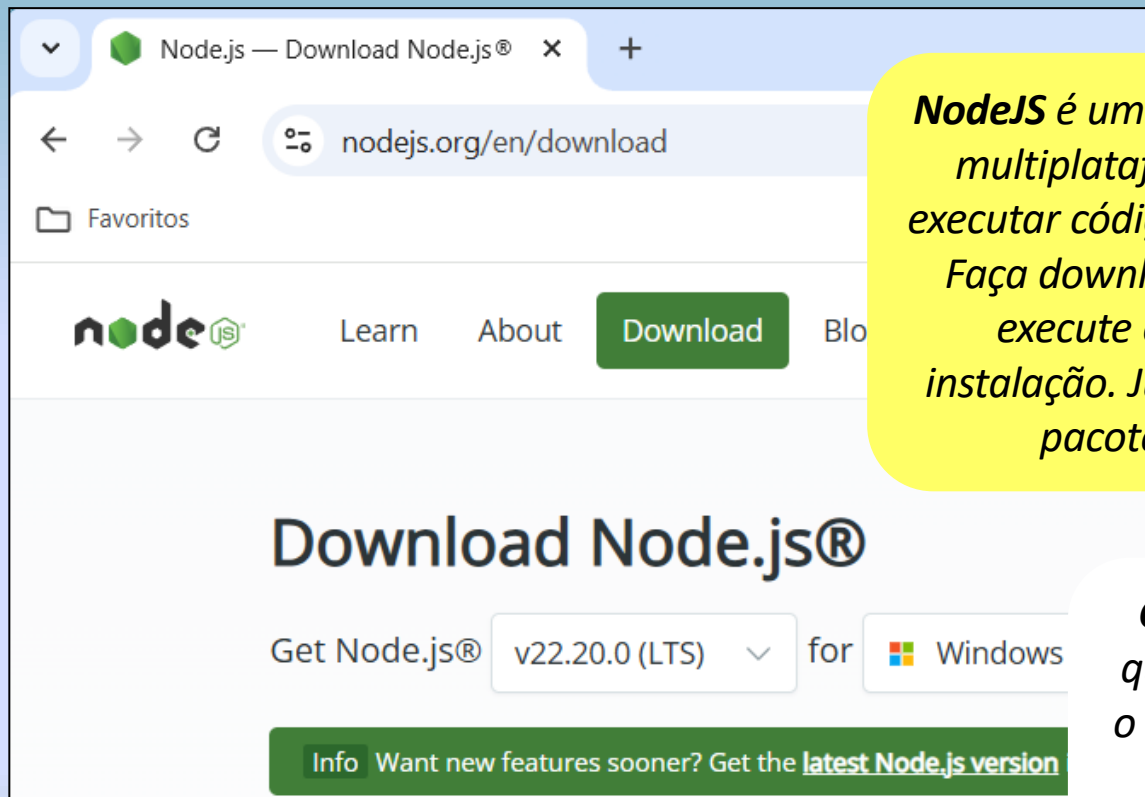
- Para inserir comentários no código JSX, use as sintaxes mostradas ao lado: “//” para comentários de uma linha e “/**/” para blocos de comentários. Observe que os comentários também devem ser envolvidos por chaves.

```
{  
  // Comentário  
}
```

```
{ /*  
  Comentários  
  Comentários  
*/ }
```

Ferramentas de Desenvolvimento

- Além do **Visual Studio Code**, para usar o React, será preciso instalar a ferramenta **NodeJS**, disponível em <https://nodejs.org/en/download>.



***NodeJS** é um ambiente de execução de código aberto e multiplataforma que permite aos desenvolvedores executar códigos JavaScript fora de um navegador web. Faça download da versão **LTS (Long-Term Support)**, execute o instalador e siga o passo a passo de instalação. Junto com o **NodeJS** vem o gerenciador de pacotes **NPM (Node Package Manager)**.*

***Gerenciador de pacotes** é uma ferramenta que visa facilitar o uso de bibliotecas durante o desenvolvimento. O **NPM** permite criar uma estrutura inicial de pastas e arquivos para uma aplicação, além de instalar, desinstalar e atualizar bibliotecas a serem usadas por ela.*

DOM e React.DOM

- O **DOM** (*Document Object Model*) é um modelo de interface genérico que possibilita a programas e scripts acessar e modificar os conteúdos e a estrutura de um documento.
- O exemplo a seguir usa o DOM para criar marcação HTML em um documento e serve de base para entender o funcionamento do React.

```
<div id="header"></div>
<script type="text/javascript">
  var h1 = document.createElement("h1");
  h1.setAttribute("class", "cabecalho");
  var texto = document.createTextNode("Cabeçalho do Site");
  h1.appendChild(texto);
  var header = document.getElementById("header");
  header.appendChild(h1);
</script>
```

- **createElement:** Cria o elemento <h1>.
- **setAttribute:** Associa o atributo/valor class="cabecalho" ao elemento <h1>.
- **createTextNode:** Cria um nó de texto.
- **appendChild:** Insere um nó de texto no elemento <h1>. Em seguida, insere o elemento <h1> no elemento <div>.

Marcação
HTML gerada
pelo script

```
<div id="header">
  <h1 class="cabecalho">Cabeçalho do Site</h1>
</div>
```

Cabeçalho do Site

DOM e React.DOM

- Agora, vamos gerar a mesma marcação HTML com uso de React:

Para usar o React, sem um gerenciador de pacotes, é preciso atrelar ao documento os scripts **react** e **react-dom**, localizados em um servidor **CDN React**, por exemplo: <https://unpkg.com/>. Para isso, basta criar dois *links* na seção **head** do documento referenciando os scripts.

*Buscar uma biblioteca em um servidor **CDN (Content Delivery Network)** pode ser vantajoso, pois, uma vez carregada, a biblioteca fica no cache do navegador. Por outro lado, buscar uma biblioteca no ambiente local sempre requer acesso ao servidor da aplicação.*

```
<head>
  <script src="https://unpkg.com/react@18.3.1/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18.3.1/umd/react-dom.development.js"></script>
</head>
<body>
  <div id="header"></div>
  <script>
    const marcacao = React.createElement("h1", {className: "cabecalho"}, "Cabeçalho do Site");
    ReactDOM.render(marcacao, document.getElementById("header"));
  </script>
</body>
```

O resultado é exatamente o mesmo do exemplo anterior.

Cabeçalho do Site

DOM e React.DOM

```
<div id="header">
  <h1 class="cabecalho">Cabeçalho do Site</h1>
</div>
```

O método **createElement** do React, usado para criar marcação HTML, admite três parâmetros: **1)** Elemento HTML a ser criado, **2)** Objeto JavaScript que define um ou mais atributos para o elemento {atributo1: "valor1", atributo2: "valor2", ...} e **3)** Conteúdo do elemento.

```
<head>
  <script src="https://unpkg.com/react@18.3.1/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18.3.1/umd/react-dom.development.js"></script>
</head>
<body>
  <div id="header"></div>
  <script>
    const marcacao = React.createElement("h1", {className: "cabecalho"}, "Cabeçalho do Site");
    ReactDOM.render(marcacao, document.getElementById("header"));
  </script>
</body>
```

Se não houver atributos a definir para o elemento, o **2º parâmetro** deverá ser vazio {} ou null. Já o **3º parâmetro** poderá ser vazio, um conteúdo textual (nó de texto do elemento) ou até mesmo outro elemento HTML criado usando a mesma sintaxe, resultando no aninhamento de elementos em dois níveis, o que pode ser estendido a tantos níveis quanto forem necessários.

*Note que foi usado **className** para definir o atributo HTML **class**, já que **class** é uma palavra reservada da JavaScript. Ao criar a marcação HTML, o mecanismo interno do React se encarrega de transformar **className** em **class**.*

DOM e React.DOM

```
<head>
  <script src="https://unpkg.com/react@18.3.1/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18.3.1/umd/react-dom.development.js"></script>
</head>
<body>
  <div id="header"></div>
  <script>
    const marcacao = React.createElement("h1", {className: "cabecalho"}, "Cabeçalho do Site");
    ReactDOM.render(marcacao, document.getElementById("header"));
  </script>
</body>
```

O método **render** do React destina-se a injetar no DOM a marcação HTML criada. Ele admite dois parâmetros: **1)** Marcação HTML (<h1> criado com o método **createElement**) e **2)** Elemento em que a marcação HTML será inserida (<div>).

Usar JavaScript para criar e injetar marcação HTML no DOM é uma tarefa possível desde os primórdios da linguagem. No entanto, React nos fornece uma série de funcionalidades que simplificam esse trabalho.

Criação de Componentes com React

- Os componentes fazem parte da essência da filosofia do React, pois permitem dividir a interface do usuário em **partes independentes e reutilizáveis**, para que se possa pensar em cada parte isoladamente.
- Conceitualmente, os componentes são **funções JavaScript**, que aceitam **propriedades** como parâmetros e retornam **elementos React**, os quais descrevem o que deve ser mostrado na tela.
- É possível definir um componente React de duas formas:

```
function Saudacao(props) {  
  return <h1>Olá {props.nome}</h1>;  
}  
  
const Saudacao = (props) => <h1>Olá {props.nome}</h1>;
```

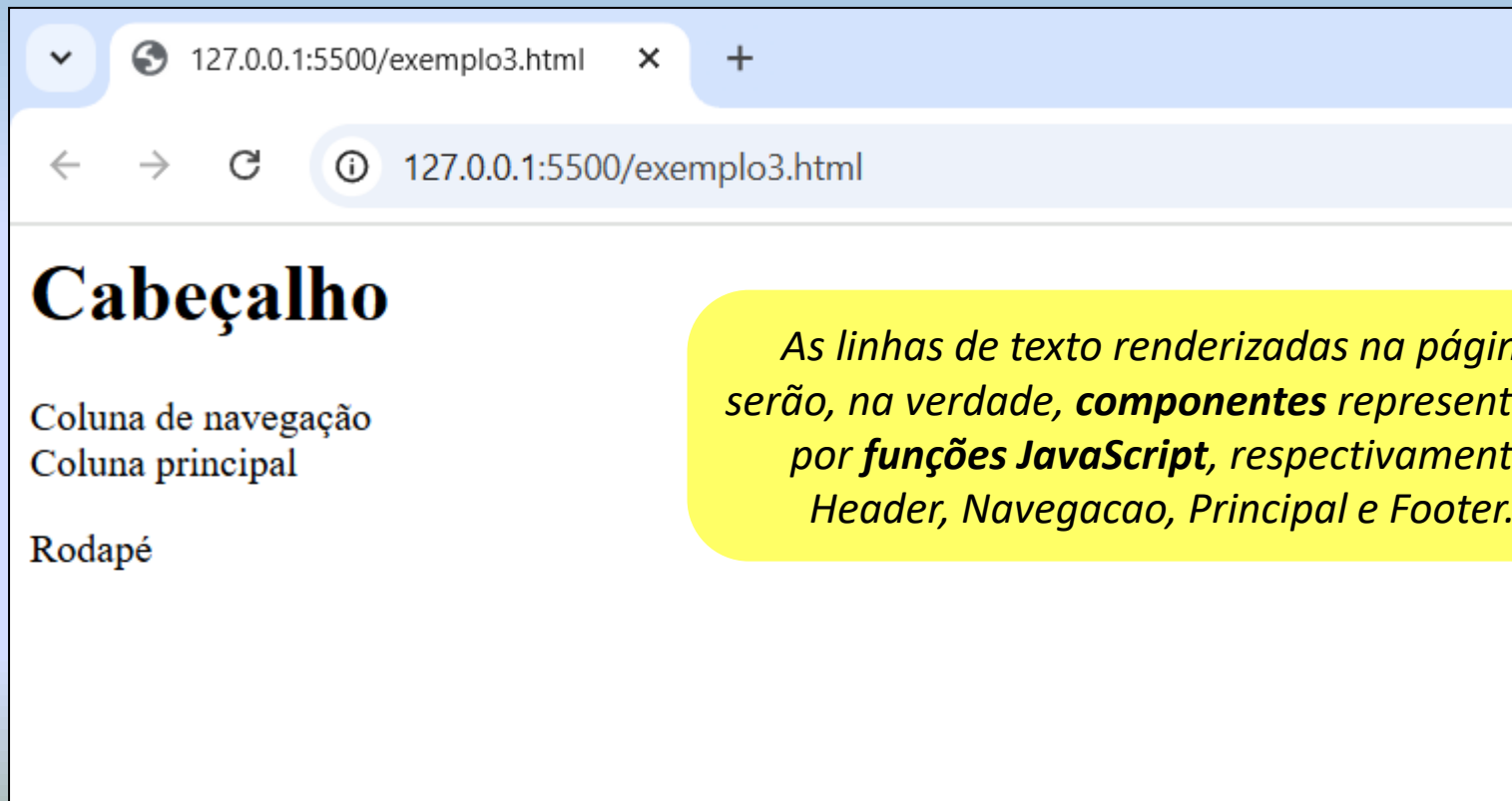
Usando uma **função JavaScript** (componente funcional) com a sintaxe ES5 ou ES6 (**arrow function**). Ou usando uma **classe** da ES6 (componente do tipo classe).

```
class Saudacao extends React.Component {  
  render() {  
    return <h1>Olá {this.props.nome}</h1>;  
  }  
}
```

Os nomes de componentes devem, obrigatoriamente, seguir a sintaxe **PascalCase**. Por exemplo: *Saudacao, CorpoTabela, RodapeTabela*.

Criação de Componentes com React

- O exemplo a seguir mostra como usar React para criar a marcação HTML de uma página Web completa, constituída por **quatro componentes**: cabeçalho, coluna de navegação, coluna de conteúdo principal e rodapé.



Criação de Componentes com React

- O código a seguir usa as funções **Header**, **Navegacao**, **Principal** e **Footer** para criar e injetar no DOM cada um dos componentes da página Web:

```
<div id="areaCabecalho"></div>
<div id="areaNavegacao"></div>
<div id="areaPrincipal"></div>
<div id="areaRodape"></div>
<script>
```

```
function Header() {
  return React.createElement("header", {className: "topo"},
    React.createElement("h1", {className: "cabecalho"}, "Cabeçalho"));
ReactDOM.render(Header(), document.getElementById("areaCabecalho"));
```

*Embora tenha sido usada a sintaxe JavaScript tradicional (ES5) para escrever as funções, uma opção preferível e mais usada atualmente é a sintaxe ES6 com **arrow functions**.*

Cria a tag <header> com um <h1> dentro.

```
function Navegacao() {
  return React.createElement("div", {className: "navegacao"}, "Coluna de navegação");
ReactDOM.render(Navegacao(), document.getElementById("areaNavegacao"));
```

Injeta a marcação criada no <div id="areaCabecalho">.

```
function Principal() {
  return React.createElement("div", {className: "principal"}, "Coluna principal");
ReactDOM.render(Principal(), document.getElementById("areaPrincipal"));
```

```
function Footer() {
  return (React.createElement("footer", {className: "rodape"},
    React.createElement("p", {}, "Rodapé")));
ReactDOM.render(Footer(), document.getElementById("areaRodape"));
```

```
</script>
```

Cabeçalho

Coluna de navegação
Coluna principal

Rodapé

Criação de Componentes com React

- No exemplo anterior, criamos e injetamos componentes React no DOM isoladamente. Porém, podemos criar um **componente-pai** para juntar os quatro componentes e injetá-los com uma única instrução.

```
<div id="tudo"></div>
<script>
  function Header() {
    return React.createElement("header", {className: "topo"},
      React.createElement("h1", {className: "cabecalho"}, "Cabeçalho"));
  }

  function Navegacao() {
    return React.createElement("div", {className: "navegacao"}, "Coluna de navegação");
  }

  function Principal() {
    return React.createElement("div", {className: "principal"}, "Coluna principal");
  }

  function Footer() {
    return (React.createElement("footer", {className: "rodape"},
      React.createElement("p", {}, "Rodapé")));
  }

  function App() {
    return React.createElement(React.Fragment, {}, Header(), Navegacao(), Principal(), Footer());
  }

  ReactDOM.render(App(), document.getElementById("tudo"));
</script>
```

O componente **App** agrupa os componentes criados Header, Navegacao, Principal e Footer. Ao final, todos são injetados no DOM da página de uma só vez.

Criação de Componentes com React

- Observe que o primeiro parâmetro do método **createElement** recebe o valor **React.Fragment**, em vez de um elemento HTML a ser criado.

```
function App() {  
  return React.createElement(React.Fragment, {}, Header(), Navegacao(), Principal(), Footer());  
ReactDOM.render(App(), document.getElementById("tudo"));
```



- Isso acontece porque, nesse código, o componente **App** será injetado no **<div id="tudo">**, sendo desnecessário criar outro elemento **<div>** para inserir os quatro componentes nele. Por exemplo:

```
function App() {  
  return React.createElement("div", {className: "app"}, Header(), Navegacao(), Principal(), Footer());  
ReactDOM.render(App(), document.getElementById("tudo"));
```



Isso serviria apenas para criar um **<div class="app">** dentro do **<div class="tudo">**.

Portanto, o elemento **React.Fragment** serve apenas para informar o 1º parâmetro do método **createElement**, que é obrigatório, mas não define nenhum elemento HTML para ser criado. Note que, como nenhum elemento é criado, o 2º parâmetro de método (atributos) fica vazio.

Ciclo de Vida dos Componentes

- O nascimento de um componente ocorre quando ele é chamado para renderização e a morte quando ele é removido da tela. No entanto, o ciclo de vida completo de um componente divide-se em quatro fases:

Initializing: Fase do nascimento

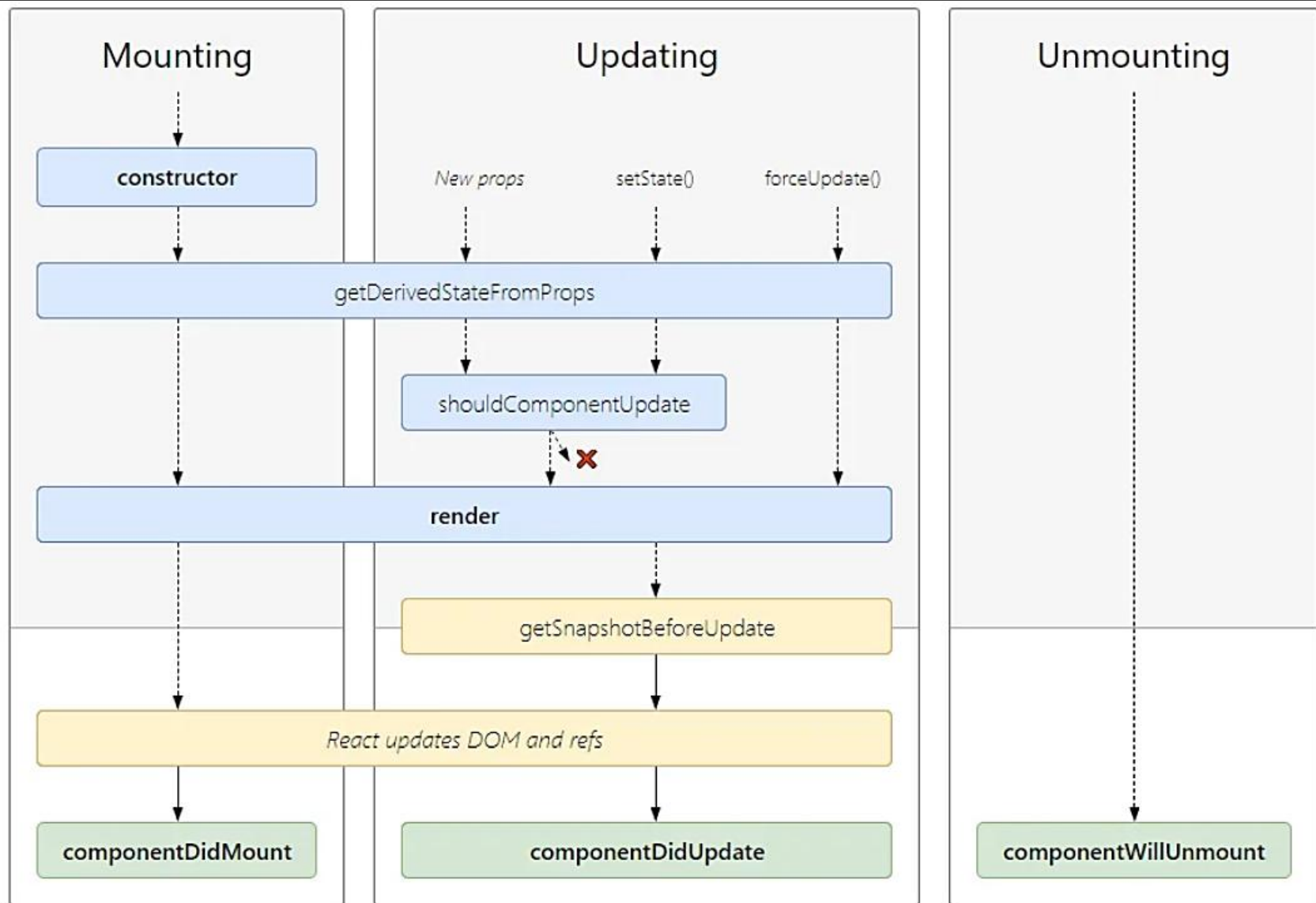
Mounting: Fase da montagem

Updating: Fase de atualizações

Unmounting: Fase da desmontagem

- Cada uma das fases possui métodos que se destinam a executar tarefas no momento mais apropriado da vida do componente.
- Muitos deles raramente são usados, outros são usados esporadicamente e outros são bastante usados.

Ciclo de Vida dos Componentes



Initializing

- Nessa fase o componente está sendo inicializado e configurado para ser renderizado.
- Apenas o método **getDerivedStateFromProps()** pode ser usado nessa fase. Ele admite dois parâmetros, **props** e **state**, que são controlados automaticamente pelo React.
- Embora esteja disponível, a própria documentação do React incentiva o desenvolvedor a não usar esse método.

```
static getDerivedStateFromProps(props, state) { }
```

Mounting

- Na fase de montagem ocorre a renderização do componente. Os dois métodos desta fase são usados com frequência:
- **render()**: Como vimos, esse método retorna um elemento React a ser carregado pelo componente. Ele é chamado sempre que o componente é carregado para renderização ou quando há alguma atualização.
- **componentDidMount()**: Esse método é chamado imediatamente após o método **render()**, ou seja, logo após ter ocorrido a renderização do componente.

*A chamada do método **componentDidMount()** é o momento ideal para execução de scripts que bloqueiam a renderização da página, pois, como a página já está renderizada, este bloqueio não ocorre. O exemplo clássico de uso desse método é para chamadas a APIs que retornam dados, as quais podem bloquear a renderização.*

Updating

- A fase de atualizações é disparada sempre que o estado de um componente muda, o que resulta em uma nova chamada do método **render()** e na chamada do método **componentDidUpdate()**, logo depois da execução de **render()**.
- Há também outros métodos, usados menos frequentemente nesta fase:

Métodos	Descrição
<code>getDerivedStateFromProps()</code>	Executado antes da renderização das atualizações do componente .
<code>shouldComponentUpdate()</code>	Executado para verificar se os valores atualizados do componente devem ser renderizados novamente. Pode ser bastante útil para otimizar performance em determinados casos quando ocorrem atualizações constantes.
<code>getSnapshotBeforeUpdate()</code>	Executado antes de uma re-renderização do componente causada por mudança. Permite a coleta de informações do DOM antes da atualização do componente , as quais poderão, por exemplo, ser passadas como parâmetros para o método componentDidUpdate() .

Unmounting

- A fase da desmontagem do componente possui apenas o método **componentWillUnmount()**.
- Ele é executado imediatamente antes de o componente ser removido do DOM.
- Pode ser útil em alguns casos em que haja necessidade de limpar a memória removendo funcionalidades tais como *timers* e *listeners* de eventos.

Referências

- KUMAR, Tejas. **React Fluente**: Crie aplicações web rápidas, eficazes e intuitivas. São Paulo: Editora Novatec, 2024.
- SILVA, Maurício S. **React: Aprenda Praticando**. São Paulo: Editora Novatec, 2021.
- TORRES, Fernando E. **Single Page Application**. São Paulo: Editora Senac, 2023.